

AI-901 - Exam Companion (Study Notes)

AI-901 - The Exam Companion

Microsoft Azure AI Fundamentals - successor to AI-900 - everything that's tested, nothing that isn't.

Snapshot in time: these notes reflect the official skills-measured outline published on Microsoft Learn as of **April 15, 2026** (the version live through mid-2026). Microsoft Foundry, Foundry Tools, and Content Understanding are fast-moving product surfaces - exact portal navigation, SDK class names, and preview/GA status can shift between exam updates. Treat product mechanics as a snapshot and verify anything you plan to *build on* against learn.microsoft.com rather than memorizing this document as gospel.

The Exam at a Glance

Exam code	AI-901
Certification	Microsoft Certified: Azure AI Fundamentals
Format	40-60 questions, single/multi-select, case studies, drag-and-drop
Duration	45 minutes
Passing score	700 / 1000
Prerequisites	None - fundamentals level
Audience	Beginning-career AI solution developers; some Python familiarity and core cloud concepts recommended
Replaces	AI-900 (Azure AI Fundamentals), retired June 30, 2026

The blueprint - where the points live

Domain	Weight
1 - Identify AI concepts and capabilities	40-45%
2 - Implement AI solutions by using Microsoft Foundry	55-60%

Two domains, not five - AI-901 folded "workloads" and "implementation" into a tighter structure than AI-900 had, and pushed noticeably more weight onto *doing* things in Foundry rather than just describing services.

DOMAIN 1 - Identify AI Concepts and Capabilities

Weight: 40-45% - Know what things are and why they matter, before you touch a keyboard

1 - Responsible AI - the six Microsoft principles - memorize

Principle	The one-line test
Fairness	Does the system treat similar people/groups similarly? (bias in training data -> biased outcomes)
Reliability & safety	Does it perform consistently, and fail safely, including under edge cases and adversarial input?
Privacy & security	Is personal data protected, minimized, and secured across the data lifecycle?
Inclusiveness	Does it work for people of varying abilities, languages, and backgrounds?
Transparency	Do people understand how/why the system made a decision, and that they're interacting with AI?
Accountability	Is there a human/organization answerable for the system's behavior, with governance in place?

Exam pattern: a scenario describes a harm (a hiring model favoring one demographic, a chatbot that won't say it's a bot, a vision model that fails on darker skin tones) and asks *which principle is at stake*. Map the harm to the principle whose one-line test it violates - don't overthink it.

2 - How generative AI models work

- A **large language model (LLM)** is trained on massive text corpora to predict the next **token** in a sequence - everything downstream (chat, summarization, code generation) is built on that core capability.
- Text is broken into **tokens** (word pieces) before processing; token count drives both cost and context-window limits.
- The **transformer architecture** and its **attention mechanism** let the model weigh the relevance of every other token when predicting the next one - this is what lets models handle long-range context (a pronoun referring back to a noun several sentences earlier, for example).
- **Embeddings** turn text (or images) into vectors of numbers positioned so that semantically similar content sits close together - the foundation of semantic search and RAG.
- **Multimodal models** accept more than one input type in the same request - text, images, audio - instead of requiring a separate model per modality.
- **Small language models (SLMs)** trade some general capability for a smaller footprint - cheaper, faster, and viable on constrained/edge hardware, useful when a task is narrow.

3 - Choosing an AI model based on capabilities

- The **model catalog** in Microsoft Foundry lists models from multiple providers (Microsoft, OpenAI, Meta, Mistral, and others) with published benchmarks, modality support, and context-window size.
- Selection factors: modality needed (text-only vs. multimodal), reasoning depth required, latency/cost budget, context-window size, and whether the task needs a large flagship model or a smaller, cheaper model is "good enough."
- **Reasoning models** spend extra inference-time computation "thinking" before answering - better for multi-step logic/math, at the cost of latency and token spend; don't reach for one on a simple lookup or classification task.

4 - Model deployment options and configuration parameters

Deployment types - memorize this table

Deployment type	Pricing model	Latency profile	Best for
Standard / Global Standard	Pay-per-token	Shared capacity, variable	Default starting point for most workloads
Global Batch	~50% cheaper than Global Standard	Asynchronous, up to 24-hour turnaround	Large, latency-insensitive offline jobs (bulk summarization, nightly scoring)
Provisioned (PTU / Global Provisioned)	Reserved capacity, purchased in Provisioned Throughput Units	Predictable, consistent	low, Steady, high-volume, latency-sensitive production traffic

Global variants route across Microsoft's global infrastructure for capacity/availability; data zone and regional variants trade some of that flexibility for data-residency guarantees.

Generation configuration parameters

Parameter	What it controls
System message / prompt	Sets the model's role, tone, and constraints before the conversation starts
Temperature	Randomness of output - low = focused/deterministic, high = more varied/creative
Top P	Nucleus sampling - restricts token choices to the smallest set whose cumulative probability exceeds P
Max tokens	Hard cap on response length (and cost)
Stop sequences	Strings that force generation to end early

Exam pattern: "the same prompt keeps producing wildly different answers and the customer wants consistency" -> lower the temperature (and/or top P), not a different model.

5 - AI workload scenarios - know which workload fits which problem

Workload	Typical scenario
Generative AI	Draft, rewrite, summarize, or converse in natural language
Agentic AI	A system plans multi-step actions and calls tools/APIs autonomously to reach a goal, not just answer one prompt
Text analysis	Extract structure/meaning from unstructured text at scale
Speech	Convert between spoken and written language, or act on voice input
Computer vision	Understand or generate images/video
Information extraction	Pull structured fields out of documents, forms, images, audio, or video

6 - Text analysis techniques

Technique	What it does
Keyword extraction (key phrase extraction)	Surfaces the main topics/phrases in a block of text
Entity detection (Named Entity Recognition)	Identifies and categorizes people, places, organizations, dates, quantities, etc.
Sentiment analysis	Scores text as positive/negative/neutral/mixed, often with a confidence score
Summarization	Condenses a long document into a shorter abstractive or extractive summary

7 - Speech recognition and speech synthesis

- **Speech recognition (speech-to-text / STT)** converts spoken audio into text - used for transcription, voice commands, and captioning.
- **Speech synthesis (text-to-speech / TTS)** converts text into natural-sounding spoken audio - customizable voice, style, and prosody.
- **Speech translation** combines recognition with translation to convert speech in one language into text or speech in another.
- Multimodal models can now accept **spoken audio directly as input** in a single request, collapsing the traditional "STT -> LLM -> TTS" three-stage pipeline into one call for simple assistant scenarios - though a dedicated Speech service is still the right call when you need fine control (custom vocabulary, precise timestamps, specific voices).

8 - Computer vision and image-generation features

Capability	What it does
Image analysis / captioning	Describes image content in natural language, tags objects, detects landmarks
Object detection	Locates and labels multiple objects with bounding boxes
Optical character recognition (OCR / Read)	Extracts printed or handwritten text from images
Face detection	Detects faces and facial attributes for verification/organization scenarios
Image generation	Produces new images from a text prompt (and can edit/vary existing images)

9 - Extracting information from text, images, audio, and video

- From **text**: entities, key phrases, sentiment, custom classification/extraction schemas.
- From **images**: OCR'd text, tagged objects, faces, and - via Content Understanding - user-defined structured fields (e.g., extracting invoice number and total from a photographed receipt).
- From **audio**: transcription, speaker diarization, and - via Content Understanding - structured fields pulled straight out of a call recording (e.g., topics discussed, sentiment, action items).
- From **video**: scene/shot segmentation, transcription of spoken audio, and structured metadata (people, objects, topics) for cataloging and search.
- The unifying idea across all four: a **schema** defines what fields you want back, and a generative model reasons over the raw content to fill that schema in - this is the core mental model for **Azure Content Understanding**, covered in depth in Domain 2.

DOMAIN 2 - Implement AI Solutions by Using Microsoft Foundry

Weight: 55-60% - The majority of the exam - expect hands-on, "which portal tab / which SDK call" questions

10 - Microsoft Foundry - the platform shape

Microsoft Foundry (the successor branding to Azure AI Studio / Azure AI Foundry) is Azure's unified platform for building, deploying, and governing AI apps and agents. The portal experience centers on:

- **Model catalog** - browse 100+ models from Microsoft, OpenAI, Meta, Mistral, and other providers, with benchmarks and modality filters.
- **Playground** - test a model or a prompt agent interactively before writing any code.
- **Deployments** - turn a catalog model into a callable endpoint, choosing a deployment type (Sec. 4).
- **Agents** - build, test, and manage agents (prompt agents and hosted agents - Sec. 14).
- **Tools tab** - discover and connect tool integrations (function calling, code interpreter, file search, and connections to external business systems) that agents can call.

Foundry Tools is the current name for the family of pre-built AI APIs formerly called Azure AI Services / Azure Cognitive Services - **Azure Language**, **Azure Speech**, **Azure Vision**, **Azure AI Translator**, **Document Intelligence**, and **Content Understanding** all now ship as Foundry Tools, callable from the same project.

11 - Prompt engineering - system vs. user prompts

- The **system prompt** sets persistent instructions: role, tone, constraints, output format - it's set once and shapes every turn.
- The **user prompt** is the individual request/question in a given turn.
- Effective prompts are specific, give the model a role, state the desired output format, and - for tricky tasks - include a small number of examples (**few-shot prompting**).
- **Exam pattern:** "responses are inconsistent in tone/format across users" -> tighten the *system* prompt, not the user prompt.

12 - Deploying and testing a model in the Foundry portal

Typical flow: **Model catalog** -> **pick a model** -> **Deploy** -> **choose deployment type & region** -> **test in the Playground** (adjust system prompt and parameters interactively) ** -> copy the endpoint + key/identity info for code**.

13 - Building a chat client with the Foundry SDK

- The Foundry SDK for Python centers on an **AIProjectClient** (from `azure-ai-projects`), constructed with the project endpoint and an Entra ID credential (`DefaultAzureCredential` or `AzureCliCredential` from `azure-identity`) - **no long-lived API keys required**.
- From the project client you retrieve an inference client (an OpenAI-compatible chat-completions client) to send messages to your deployed model and get a response back - the same request/response chat-completions shape used across the industry (system message + user message list -> assistant reply).
- This keyless, identity-based pattern mirrors the "keyless > vaulted key > env key" preference tested elsewhere in Azure AI exams (see [14-azure-ai103/EXAM_NOTES.md](#) Sec. 3 for the deeper authentication comparison).

14 - Creating and testing a single-agent solution in the Foundry portal

Foundry Agent Service offers two agent shapes:

Agent type	How it runs	When to use
Prompt agent	Authored in the portal (or via SDK/REST) - Foundry runs it; Fast iteration, simple tool-using no application code, containers, assistants or compute to manage	
Hosted agent	You write the agent's code (with an agent framework, or your Custom orchestration logic, own), package it as a container, existing agent-framework code, and Foundry runs it with a more control managed endpoint	

A single-agent solution is created, given instructions and tools in the portal, tested directly in the Playground, and - once satisfied - called from a lightweight client the same way a chat deployment is called.

15 - Building a lightweight client application for an agent

A minimal agent client: authenticate with an Entra ID credential -> get the project's agent client -> create (or reuse) a **thread** for the conversation -> post a user message to the thread -> run the agent -> poll/stream for completion -> read the assistant's response from the thread. The **thread** is what gives an agent conversational memory across turns, distinct from a one-shot chat-completions call.

16 - Implementing AI solutions for text analysis

A lightweight text-analysis app calls **Azure Language in Foundry Tools** (or a deployed LLM prompted for the same task) to run key-phrase extraction, entity recognition, sentiment analysis, or summarization over input text, then surfaces the structured result (entities, scores, summary) to the user or downstream system.

17 - Responding to spoken prompts and using Azure Speech in Foundry Tools

- A **deployed multimodal model** can accept an audio clip directly as part of the request and return a text (or audio) response - useful for simple "talk to the assistant" scenarios without wiring up a separate Speech resource.
- **Azure Speech in Foundry Tools** is the dedicated service for when you need first-class speech features: accurate speech-to-text with custom vocabulary, specific synthesized voices/styles for text-to-speech, or speech translation - a lightweight app typically calls Speech to transcribe input, optionally routes the text through a model, and calls Speech again to synthesize a spoken reply.

18 - Computer vision and image generation in Foundry

- A **deployed multimodal model** can accept an image alongside a text prompt and answer questions about what's in the image ("interpreting visual input") - no separate vision resource required for many multimodal Q&A scenarios.
- Generating **new visual outputs** (an image from a text description, or a variation of an existing image) requires an image-generation model rather than an image-understanding model - know the distinction between "understand this image" and "create an image."
- A lightweight vision app wires a deployed multimodal or image-generation model into a simple UI/API that accepts an image or prompt and returns the analysis or the generated image.

19 - Information extraction with Azure Content Understanding

- **Azure Content Understanding in Foundry Tools** uses generative AI to turn unstructured

documents, images, audio, and video into a **user-defined structured output** (a schema you specify: field names, descriptions, and types) - reached GA with the 2025-11-01 API version.

- **Analyzers** are content-type specific:
- **Document analyzer** - extracts fields from forms/documents; complements **Document Intelligence** (deterministic, high-accuracy extraction from well-structured documents) with LLM-powered handling of messier, multimodal content.
- **Image analyzer** - schema-driven field extraction from photos (e.g., pull a serial number and condition rating out of a product photo).
- **Audio analyzer** - structured fields from call/meeting recordings (topics, sentiment, action items) on top of transcription.
- **Video analyzer** - scene segmentation plus custom structured metadata for cataloging and search.
- **Standard vs. Pro modes** trade off cost/speed against extraction depth for complex content - pick Pro when a schema needs deeper reasoning over the source material, Standard for straightforward, well-defined fields.
- A lightweight extraction app defines a schema, calls the appropriate Content Understanding analyzer with the source file, and consumes the structured JSON result - the same "define the shape you want, let the model fill it in" pattern across all four content types.

QUICK RECAP - Your Exam-Week Arsenal

20 - Rapid-fire tables

Which deployment type?

Signal in the question	Answer
"steady high volume, needs predictable low latency"	Provisioned (PTU)
"large overnight/offline batch job, cost matters more than speed"	Global Batch
"just getting started / typical interactive workload"	Standard / Global Standard

Which Foundry agent type?

Signal in the question	Answer
"no application code to maintain, build it in the portal"	Prompt agent
"we already have custom orchestration code / a specific agent framework"	Hosted agent

Which RAI principle?

Signal in the question	Principle
Model performs worse for one demographic group	Fairness
System fails ungracefully on unusual/edge-case input	Reliability & safety
Personal data over-collected or exposed	Privacy & security
Product doesn't work for users with disabilities/other languages	Inclusiveness

Signal in the question	Principle
Users don't know they're talking to a bot / can't tell why a decision was made	Transparency
No one is answerable when the system causes harm	Accountability

Which workload?

Signal in the question	Workload
"summarize/rewrite/draft in natural language"	Generative AI
"plans steps and calls tools on its own toward a goal"	Agentic AI
"pull entities/sentiment/keywords out of text"	Text analysis
"convert speech <-> text"	Speech
"understand or create images/video"	Computer vision
"turn a document/photo/recording into structured fields"	Information extraction

21 - Flash facts - read 10 minutes before the exam

- Two domains only: **40-45% concepts, 55-60% Foundry implementation** - expect more "which portal step / which SDK call" questions than AI-900 had.
- Six RAI principles, in the exam's own order: **fairness, reliability & safety, privacy & security, inclusiveness, transparency, accountability.**
- Three deployment types: **Standard/Global Standard -> Global Batch -> Provisioned (PTU)**, in order of "cheap & shared" to "reserved & predictable."
- **Prompt agents** = no code, portal-authored. **Hosted agents** = your code, containerized, Foundry-run.
- Foundry SDK auth is **keyless** - `AIPProjectClient + DefaultAzureCredential/AzureCliCredential`, no API keys in the happy path.
- **Foundry Tools** = the new name for Azure AI Services (Language, Speech, Vision, Translator, Document Intelligence, Content Understanding).
- **Content Understanding** = schema in, structured JSON out, across documents/images/audio/video - the one workload that spans all four content types.
- Multimodal models can take **image or audio directly as input** in one call - don't assume you always need a separate Vision or Speech resource for simple understanding tasks; you do need one for speech *synthesis* or generative *image creation*.

See also

- [EXAM_PRACTICE_QUESTIONS.md](#) - original scenario-based practice questions built from this document
- [14-azure-ai103/EXAM_NOTES.md](#) - the associate-level follow-on exam, if you're continuing past fundamentals
- [11 azure ai foundry/](#) - hands-on labs against a real Foundry project (agents, tools, RAG)